

NAME:

STD.:

SEC.:

ROLL NO.:

SUB.:

Index

* Aim :- To design, stimulate and verify the magnification functionality of various fundamental combinational logic circuits using Verilog HDL.

* Theory :-

This experiment covers the Design and verification of standard combinational circuits and fundamental concepts of the Verilog language.

* Tools Required :-

① Model Sim :- This is an HDL simulation tool used for verifying the functionality of digital circuits.

② Verilog HDL :- This is the language used to describe the digital circuits and their test benches.

Date
12-Aug-25

{ Assessment - 2 }

classmate
Date
Page

Full Adder

```
module Full-adder (a, b, cin, Sum, Carry, d, bo);  
    input a, b, cin;  
    output Sum, Carry;  
    output d, bo;  
    assign Sum = a ^ b ^ cin;  
    assign Carry = (a & b) | cin & (a ^ b);  
    assign d = a ^ b ^ cin;  
    assign bo = ((~a) & b) | ((a ^ b) & (~cin));  
endmodule
```

```
module tb();
```

```
    reg a, b, cin;
```

```
    wire Sum, Carry;
```

```
    wire d, bo;
```

```
    Full_Adder m0 (a, b, cin, Sum, Carry, d, bo);
```

```
    initial
```

```
    begin
```

```
        a = 1'b0; b = 1'b0; cin = 1'b0;
```

```
        #100 a = 1'b0; b = 1'b0; cin = 1'b0;
```

```
        #100 a = 1'b0; b = 1'b0; cin = 1'b0;
```

```
        #100 a = 1'b0; b = 1'b0; cin = 1'b0;
```

```
        #100 a = 1'b0; b = 1'b0; cin = 1'b0;
```

```
        #100 a = 1'b0; b = 1'b0; cin = 1'b0;
```

```
        #100 a = 1'b0; b = 1'b0; cin = 1'b0;
```

```
        #100 a = 1'b0; b = 1'b0; cin = 1'b0;
```

```
        #100 a = 1'b0; b = 1'b0; cin = 1'b0;
```

```
    end
```

```
endmodule
```

o/p verified

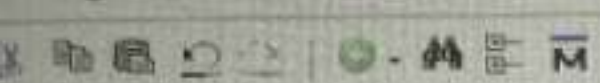
12/8/25
24 BCE 0403


```

1 module Full_Adder(a,b,cin,sum,carry,d,b0);
2   input a,b,cin;
3   output sum,carry;
4   output d,b0;
5   assign sum=a^b^cin;
6   assign carry=(a&b)|cin&(a^b);
7   assign d=a^b^cin;
8   assign b0=((~a)&b)|((a^b)&(~cin));
9 endmodule
10 module tb();
11   reg a,b,cin;
12   wire sum,carry;
13   wire d,b0;
14   Full_Adder mo(a,b,cin,sum,carry,d,b0);
15   initial
16   begin
17     a=1'b0;b=1'b0;cin=1'b0;
18     #100
19     a=1'b0;b=1'b0;cin=1'b0;
20     #100
21     a=1'b0;b=1'b0;cin=1'b0;
22     #100
23     a=1'b0;b=1'b0;cin=1'b0;
24     #100
25     a=1'b0;b=1'b0;cin=1'b0;
26     #100
27     a=1'b0;b=1'b0;cin=1'b0;
28     #100
29     a=1'b0;b=1'b0;cin=1'b0;
30     #100
31     a=1'b0;b=1'b0;cin=1'b0;
32     #100
33     a=1'b0;b=1'b0;cin=1'b0;
34   end
35 endmodule
36
37

```





Layout Simulate

ColumnLayout AllColumns

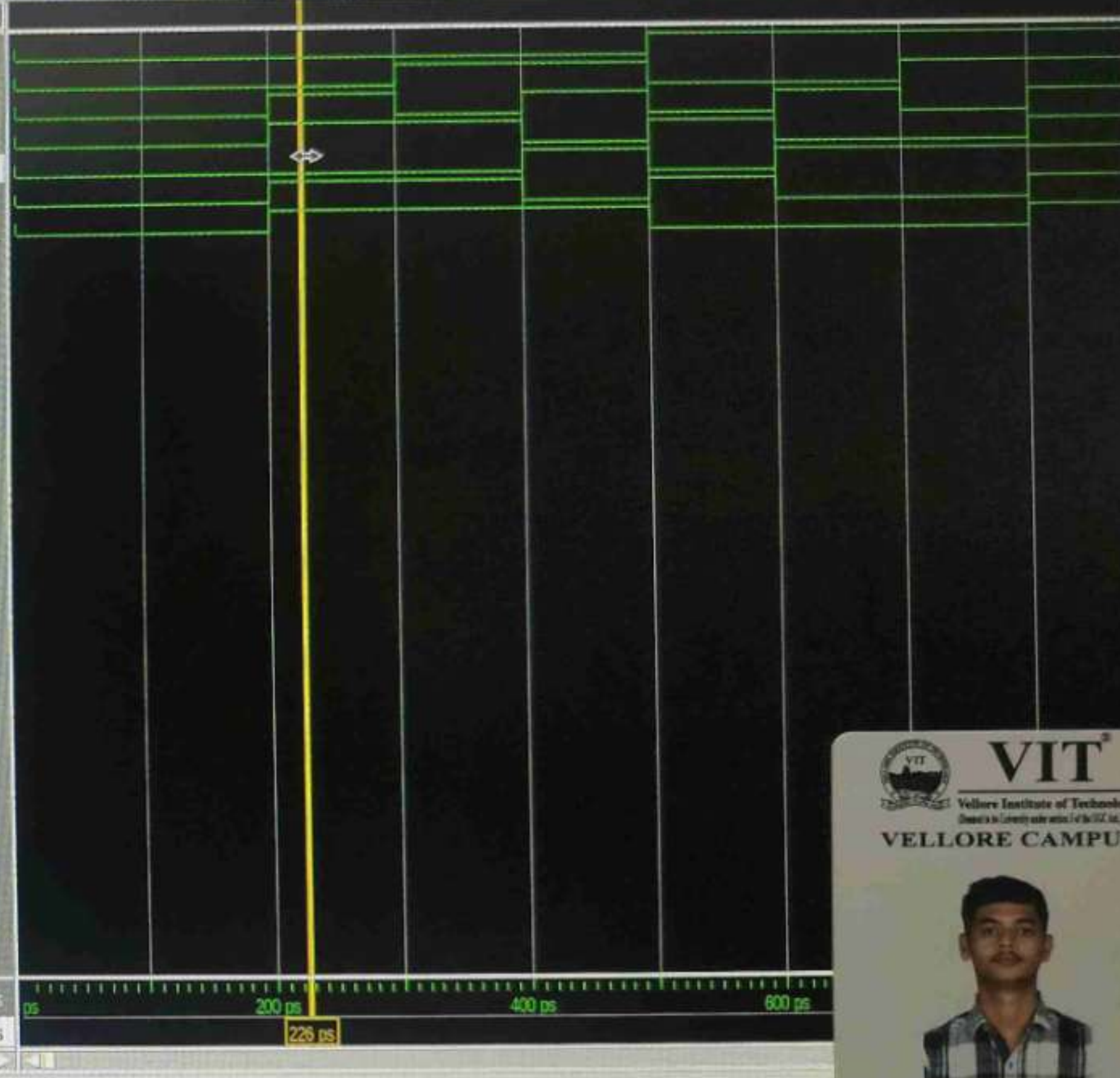


100 ns

```
1 module FullAdder(a,b,cin,sum,carry,d,b0);
2   input a,b,cin;
3   output sum,carry;
4   output d,b0;
5   assign sum=a^b^cin;
6   assign carry=(a&b)|cin&(a^b);
7   assign d=a^b^cin;
8   assign b0=(-a&b)|((cin & (-(a^b))));
9 endmodule
10 module tb();
11   reg a,b,cin;
12   wire sum,carry;
13   wire d,b0;
14   FullAdder mo(a,b,cin,sum,carry,d,b0);
15   initial
16   begin
17     a=1'b0;b=1'b0;cin=1'b0;
18     #100a=1'b0;b=1'b0;cin=1'b0;
19     #100a=1'b0;b=1'b0;cin=1'b1;
20     #100a=1'b0;b=1'b1;cin=1'b0;
21     #100a=1'b0;b=1'b1;cin=1'b1;
22     #100a=1'b1;b=1'b0;cin=1'b0;
23     #100a=1'b1;b=1'b0;cin=1'b1;
24     #100a=1'b1;b=1'b1;cin=1'b0;
25     #100a=1'b1;b=1'b1;cin=1'b1;
26   end
27 endmodule
28
```

Wave Default

Signal	Value
tb/a	0
tb/b	0
tb/cin	1
tb/sum	S0
tb/carry	S0
tb/d	S0
tb/b0	S0

er
ertpoint sim:/tb/*

Project: taladitya Now: 100 ns Delta: 0

tb/sum



Aryan Singh Sikarv

24BCE0403

HOSTELLER

$\left. \begin{matrix} T \rightarrow 0 \\ F \rightarrow 1 \end{matrix} \right\}$

A	B	C	D	Bo	
T	T	T	T	T	
T	T	F	F	F	
T	F	T	F	F	
T	F	F	T	F	
F	T	T	F	T	
F	T	F	T	T	
F	F	T	T	T	
F	F	F	F	F	

#2 Decoder Test Bench code :-

```

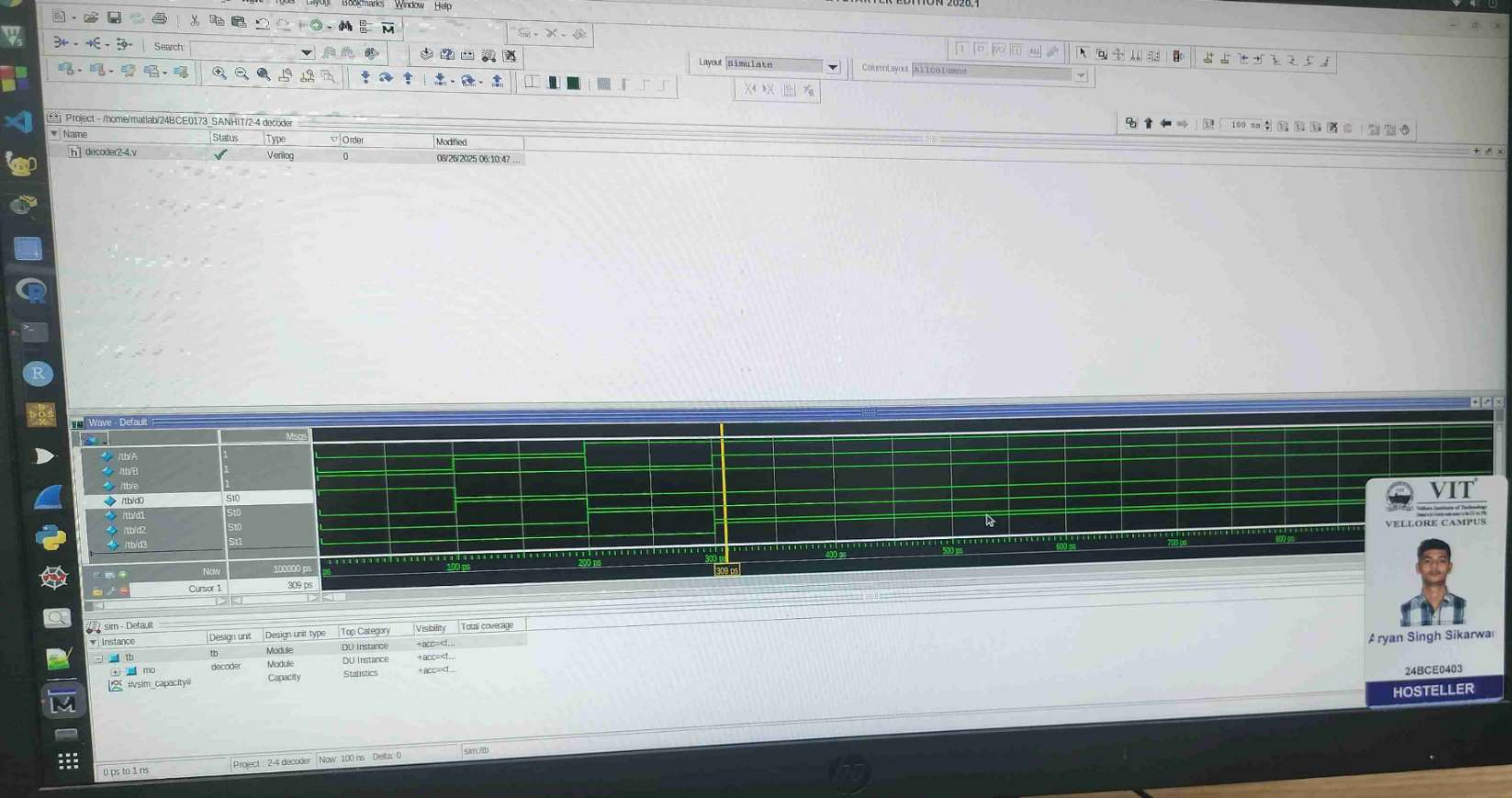
module decoder (A, B, e, d0, d1, d2, d3);
    input A, B;
    output d0, d1, d2, d3;
    assign d0 = ~A & ~B & e;
    assign d1 = ~A & B & e;
    assign d2 = A & ~B & e;
    assign d3 = A & B & e;
endmodule tb();

sig A, B, e;
wire d0, d1, d2, d3;
decoder dec (A, B, e, d0, d1, d2, d3);

initial
begin
    A = 0'b0; B = 0'b0; e = 1'b1;
    #100 A = 1'b0; B = 1'b0; e = 1'b1;
    #100 A = 1'b1; B = 1'b0; e = 1'b1;
    #100 A = 1'b1; B = 1'b1; e = 1'b1;
end
endmodule

```

o/p verified
 26/8/25
 24050403



Multiplexer Code :-

```

module Multiplexer (d0, d1, d2, d3, d4, d5,
    , d6, d7, sel, out) ;
    input d0, d1, d2, d3, d4, d5, d6, d7;
    input [2:0] sel;
    output reg out;
    always @ (sel)
    begin
        case (sel)
            3'b000 : out = d0;
            3'b001 : out = d1;
            3'b010 : out = d2;
            3'b011 : out = d3;
            3'b100 : out = d4;
            3'b101 : out = d5;
            3'b110 : out = d6;
            3'b111 : out = d7;
        endcase
    end
endmodule

```

of verified
 26/05/25
 24BCE0403

```

module Testbench ;
    reg d0, d1, d2, d3, d4, d5, d6, d7;
    reg [2:0] sel;
    wire out;
    multiplexer uut (.d0(d0), .d1(d1), .d2(d2),
        .d3(d3), .d4(d4), .d5(d5), .d6(d6),
        .d7(d7), .sel(sel), .out(out));
    initial begin
        d0 = 0; d1 = 0; d2 = 0; d3 = 0; d4 = 0;
        d5 = 0; d6 = 0; d7 = 0; sel = 0;
        # 100;
        d0 = 0; d1 = 0; d2 = 0; d3 = 0;
        d4 = 1; d5 = 1; d6 = 0; d7 = 1;
    end
endmodule

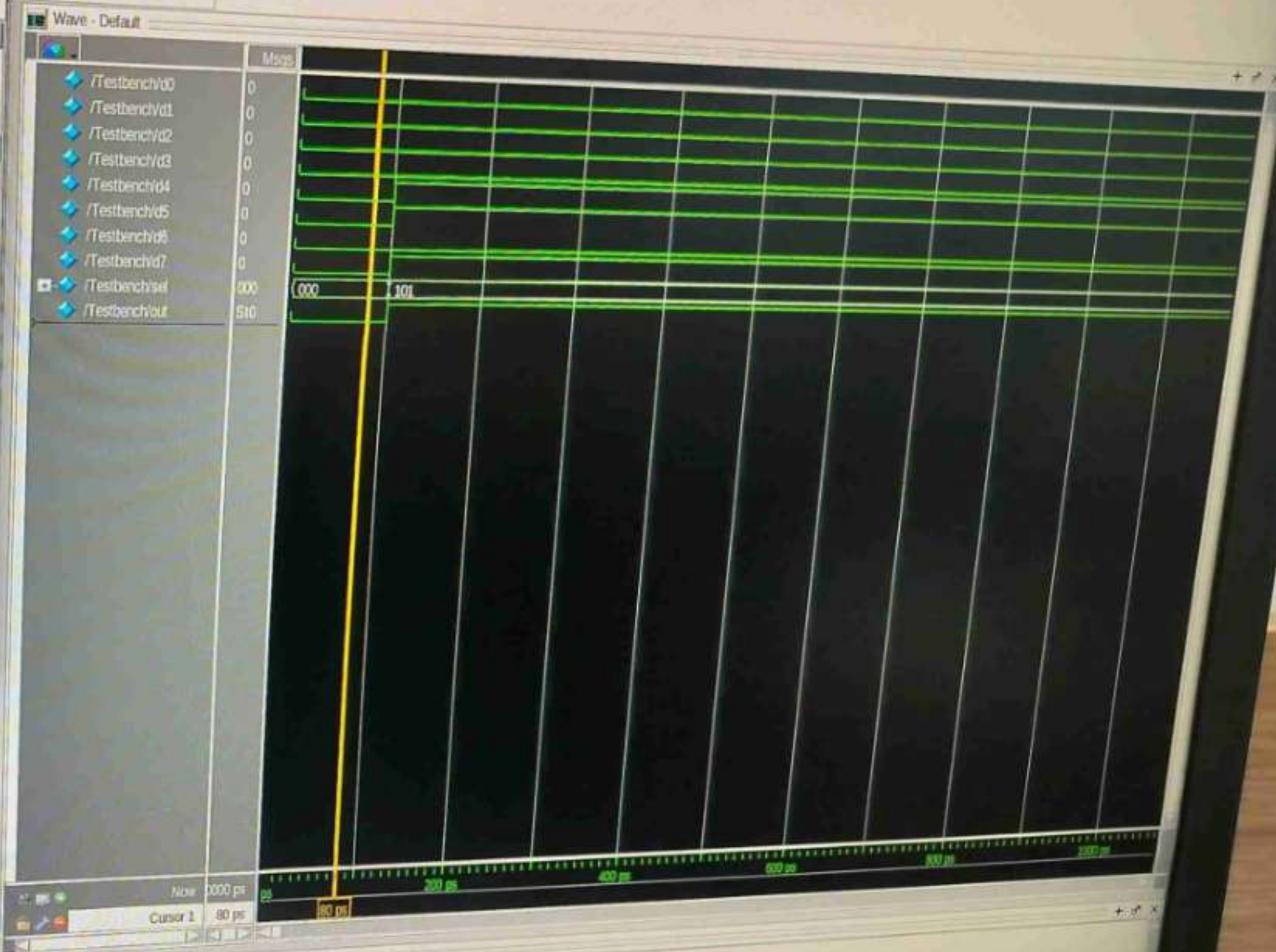
```


/home/matlab/Arya_24bce2761/2532.v (/Testbench)

File Edit View Tools Bookmarks Window Help

/home/matlab/Arya_24bce2761/2532.v (/Testbench) - Default

```
1 module Multiplexer(d0,d1,d2,d3,d4,d5,d6,d7,sel,out);
2 input d0,d1,d2,d3,d4,d5,d6,d7;
3 input [2:0]sel;
4 output reg out;
5 always @(sel)
6 begin
7 case(sel)
8 3'b000:out=d0;
9 3'b001:out=d1;
10 3'b010:out=d2;
11 3'b011:out=d3;
12 3'b100:out=d4;
13 3'b101:out=d5;
14 3'b110:out=d6;
15 3'b111:out=d7;
16 endcase
17 end
18 endmodule
19
```



Demultiplexer Code 8-

```
module demultiplexer (a,x,y,z,d0,d1,d2,  
d3,d4,d5,d6,d7);
```

```
input a,x,y,z;
```

```
output d0,d1,d2,d3,d4,d5,d6,d7;
```

```
assign d0 = (a & ~x & ~y & ~z);
```

```
d1 = (a & ~x & ~y & z);
```

```
d2 = (a & ~x & y & ~z);
```

```
d3 = (a & ~x & y & z);
```

```
d4 = (a & x & ~y & ~z);
```

```
d5 = (a & x & ~y & z);
```

```
d6 = (a & x & y & ~z);
```

```
d7 = (a & x & y & z);
```

```
endmodule
```

```
module Testbench - demux;
```

```
reg a,x,y,z;
```

```
wire d0,d1,d2,d3,d4,d5,d6,d7;
```

```
demultiplexer uut (.a(a), .x(x), .y(y),
```

```
.z(z), .d0(d0), .d1(d1), .d2(d2), .d3(d3),
```

```
.d4(d4), .d5(d5), .d6(d6), .d7(d7));
```

```
initial
```

```
begin
```

```
a=0; x=0; y=0; z=0;
```

```
#100 a=1; x=0; y=1; z=0;
```

```
#100 a=1; x=1; y=0; z=0;
```

```
#100 a=1; x=1; y=1; z=1;
```

```
end
```

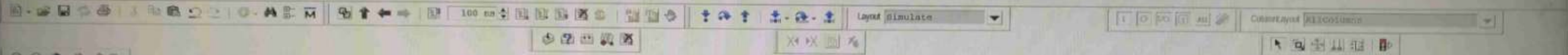
```
endmodule
```

off Verified

26/08/25

24BCE0403

File Edit View Compile Simulate Add Wave Tools Layout Bookmarks Window Help



Instance	Design unit	Design unit type	Top Category	Visibility	Total coverage
tb	tb	Module	DU Instance	+acc=d...	
de_mux	de_mux	Module	DU Instance	+acc=d...	
avsim_capacity		Capacity	Statistics	+acc=d...	

h:\home\matlab\rayn_mux.v (tb) - Default

```
1 module de_mux(a,x,y,z,d0,d1,d2,d3,d4,d5,d6,d7);
2   input a,x,y,z;
3   output d0,d1,d2,d3,d4,d5,d6,d7;
4   assign d0 = (a&-x&-y&-z),
5   d1 = (a&-x&-y&z),
6   d2 = (a&-x&y&-z),
7   d3 = (a&-x&y&z),
8   d4 = (a&x&y&-z),
9   d5 = (a&x&y&z),
10  d6 = (a&x&y&-z),
11  d7 = (a&x&y&z);
12 endmodule
13 module tb;
14   reg a,x,y,z;
15   wire d0,d1,d2,d3,d4,d5,d6,d7;
16   de_mux uut(.a(a),.x(x),.y(y),.z(z),.d0(d0),.d1(d1),.d2(d2),.d3(d3),.d4(d4),.d5(d5),.d6(d6),.d7(d7));
17   initial begin
18     a=0;x=0;y=0;z=0;
19     #100 a=1;x=0;y=1;z=0;
20     #100 a=1;x=1;y=0;z=0;
21     #100 a=1;x=1;y=1;z=1;
22   end
23 endmodule
24
```

VIT
Vellore Institute of Technology
VELLORE CAMPUS



Aryan Singh Sikarwar

24BCE0403

HOSTELLER

```
0 (50 ps) -r 100
d with 3 errors.
d with 2 errors.
d with 5 errors.
successful.
d with 5 errors.
successful.
successful.
6,2025, Elapsed time: 0:24:58
26,2025
e sims/tb/*
```

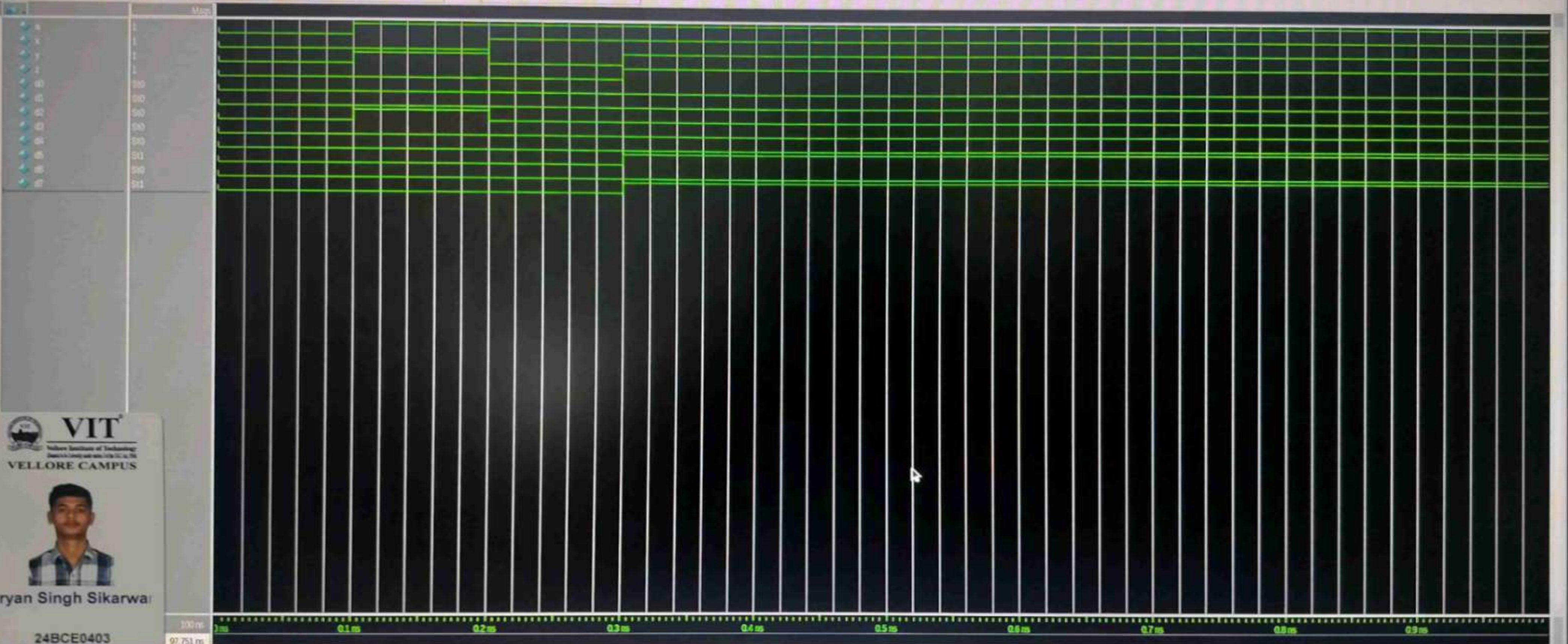
max: Now 100 ns Data: 0

sim: 0

Wave

File Edit View Add Format Tools Bookmarks Window Help

File View Default



VIT
Vellore Institute of Technology
VELLORE CAMPUS



Aryan Singh Sikarwar

24BCE0403

HOSTELLER

100 ms
97.751 ms

Top Category	Visibility	Total coverage
DU Instance	+acc<f...	
DU Instance	+acc<f...	
Statistics	+acc<f...	

```
1 module decorder(A,B,e,d0,d1,d2,d3);
2   input A,B,e;
3   output d0,d1,d2,d3;
4   assign d0 = ~A & ~B & e;
5   assign d1 = ~A & B & e;
6   assign d2 = A & ~B & e;
7   assign d3 = A & B & e;
8 endmodule
9
10 module tb();
11   reg A,B,e;
12   wire d0,d1,d2,d3;
13   decorder dec(A,B,e,d0,d1,d2,d3);
14   initial
15   begin
16     A=1'b0;B=1'b0;e=1'b1;
17     #100 A = 1'b0;B = 1'b1;e=1'b1;
18     #100 A = 1'b1;B = 1'b0;e=1'b1;
19     #100 A = 1'b1;B = 1'b1;e=1'b1;
20   end
21 endmodule
22
```

Search For

Aa

(a)

13

Wave

h

defau v

Answer the following

* what do you mean by test Bench in Verilog?

Ans It creates a copy of the module you want to test and places it inside the virtual lab. It separates piece of a code you write to check if your main hardware design.

* what is Difference between initial and always?

Ans The initial block runs only piece at the beginning of a simulation typically for setting up a test. The always block runs repeatedly and continuously, modeling the behaviour of real hardware always active.

* what is instantiation in Verilog?

Ans Instantiation is the process of creating a usable copy of one module inside another. It is the fundamental method for building complex, hierarchical circuits from smaller, reusable components.